EECS 348 Group 10

Arithmetic Expression Evaluator
Software Requirements Specifications

Version 1.0

Revision History

Date	Version	Description	Author
03/10/26	1.0	First additions to sections 2.1.5 -> 2.5	Tate Meyer
03/11/26	1.0 1.1	Sections 2-2.1.4 are completed Fixed spelling, grammar, indentation, and formatting for 2-2.1.4	Jaycob Campos
03/11/2026	1.2	Sections 1.1,1.2 and 1.3 are completed	Nikki Sotoodi
03/13/2026	1.3	Sections 3 and 4 are completed	Avery Richardson
3/15/2026	1.4	Section completed 1.4 + introduction, completed section 5	Nikki Sotoodi

Table of Contents

1. Introduction 4
   - 1.1 Purpose 4
   - 1.2 Scope 4
   - 1.3 Definitions, Acronyms, and Abbreviations 4
   - 1.4 References 5
   - 1.5 Overview 5

2. Overall Description 5
   - 2.1 Product perspective 5
     - 2.1.1 System Interfaces 6
     - 2.1.2 User Interfaces 6
     - 2.1.3 Hardware Interfaces 6
     - 2.1.4 Software Interfaces 6
     - 2.1.5 Communication Interfaces 6
     - 2.1.6 Memory Constraints 6
     - 2.1.7 Operations 6
   - 2.2 Product functions 6
   - 2.3 User characteristics 7
   - 2.4 Constraints 7
   - 2.5 Assumptions and dependencies 7
   - 2.6 Requirements subsets 7

3. Specific Requirements 7
   - 3.1 Functionality 7
     - 3.1.1 <Functional Requirement One> 7
     - 3.1.2 <Functional Requirement Two> 6
     - 3.1.3 <Functional Requirement Three> 6
   - 3.2 Use-Case Specifications 8
   - 3.3 Supplementary Requirements 8

4. Classification of Functional Requirements 9

5. Appendices 9

Software Requirements Specifications

1. Introduction

This document describes the Software Requirements Specification for the Arithmetic Expression Evaluator project. It provides an overview of the system, its purpose, and the requirements needed for development. The document is intended to help the development team understand the expected behavior of the software and provide a clear reference during the design and implementation process.

1.1 Purpose

The purpose of this Software Requirements Specification is to describe the requirements for the Arithmetic Expression Evaluator system. This document explains what the system is supposed to do, the features it must support, and any limitations or constraints of the program. It provides a clear description of the expected behavior of the software so that developers and testers understand how the system should function.

1.2 Scope

This project involves the development of an Arithmetic Expression Evaluator implemented in C++. The program will allow users to enter arithmetic expressions containing numeric values and mathematical operators.

The evaluator will support the operators +, -, *, /, %, and exponentiation (**), as well as unary + and -. The system will follow standard mathematical precedence rules and correctly handle parentheses when evaluating expressions.

The program will also detect invalid expressions and provide error messages for issues such as incorrect syntax or division by zero. The evaluator will run as a command-line application and serve as a component that could later be integrated into a larger compiler system.

1.3 Definitions, Acronyms, and Abbreviations

SRS (Software Requirements Specification): A document that describes the requirements and behavior of a software system.

Parser: A component of a program that analyzes an input expression and determines its structure based on defined rules.

Token: A small unit of an expression, such as a number, operator, or parenthesis.

Operator: A mathematical symbol used to perform a calculation, such as +, -, *, /, %, or **.

PEMDAS: The standard order of operations used in mathematics: Parentheses, Exponents, Multiplication/Division, Addition/Subtraction.

Expression: A sequence of numbers, operators, and parentheses that can be evaluated to produce a numerical result.

1.4 References

- EECS 348 course materials, University of Kansas, Spring 2026.
- EECS 348 Term Project Description – Arithmetic Expression Evaluator in C++ , published by KU EECS Department

1.5 Overview

This document is organized into 5 sections in the following order:

1. Introduction: An overview of the SRS, including purpose, scope, definitions, and references.
2. Overall Description: Describes general factors and the background of requirements that will be defined in section 3.
3. Specific Requirements: Describes software requirements and specifications.
4. Classification of functional requirements: A table listing all the requirements and their level of importance to the final product.
5. Appendices: Any additional information required.

2. Overall Description

The Arithmetic Expression Evaluator is in C++ and will be developed as a parser component for a larger compiler product for language L. It takes arithmetic expressions as its input from a user and parses them according to mathematical rules. Then output the computed result. The program will support the operators +, -, *, /, %, and **(exponentiation). As well as unary + and – operators and parenthesized sub-expressions. Operands are limited to integers and floats.

This section describes the general factors that affect the product and its requirements, including user interface characteristics, hardware, and software dependencies. Specific functional requirements are defined in detail in Section 3.

2.1 Product perspective

The Arithmetic Expression Evaluator is a self-contained component of a larger compiler L, specifically designed as the expression parser and evaluation module for the compiler L. The evaluator must conform to the operator rules defined by L. These standards follow PEMDAS convention with left-to-right associativity for +, -, *, /, %, and **(exponentiation). It will have a right-to-left associativity for ** (exponentiation). It will support +, %, and **(exponentiation).

In the development phase, the evaluator will operate independently, allowing for testing and validation before integration into L.

2.1.1 System Interfaces

The evaluator is designed to integrate with the compiler for language L as an internal module. But by itself it uses the host operating system's standard output and input streams. When integrated into the compiler, the evaluator will expose a function-based interface.

2.1.2 User Interfaces

The program will prompt the user to enter an arithmetic expression. After the user submits an expression, the program will display either the computed result or an error message describing the problem. The interface will allow for multiple evaluations until the user chooses to exit.

2.1.3 Hardware Interfaces

The evaluator does not interact directly with any specialized hardware. It requires a computer capable of running a C++ compiled executable. The program will use the keyboard for input and the console for output.

2.1.4 Software Interfaces

The program will be compiled using a C++ compiler supporting C++17 or later. It relies on the built-in library for string manipulation, data structures, and console input and output.

2.1.5 Communication Interfaces

The Arithmetic Expression Evaluator operates entirely as a standalone command-line application and does not require any network or inter-process communication interfaces. All input is received directly from the user via standard input, and all output is written to standard output or standard error.

2.1.6 Memory Constraints

The application imposes no significant memory constraints beyond that of a normal desktop or laptop computer. Because all expression processing occurs in-memory using a recursive parsing approach and operator stacks, memory usage would only scale with expression complexity. The program must be capable of processing expressions of reasonable length and depth on any machine meeting the minimum system requirements. No persistent memory storage is required; all states are held only for the duration of a single cycle.

2.1.7 Operations

The system supports a single interactive operation mode, once the user launches the program from a command-line environment, enters an arithmetic expression, and receives the evaluated result or an error message. The program does not require administration, scheduled tasks, or background services. Each execution run is self-contained: the user may evaluate one or more expressions in sequence, and the program terminates when the user exits.

2.2 Product functions

The Arithmetic Expression Evaluator provides the following primary functions:

- **Expression Input**: Accept a user-supplied arithmetic expression as a string via the command-line interface.
- **Tokenization**: Break the input string into a sequence of tokens representing numeric constants, operators (+, %, **), parentheses, and unary operators.
- **Parsing**: Construct an internal representation of the expression that captures operator precedence, associativity, and grouping with parentheses according to PEMDAS rules.
- **Evaluation**: Compute the numerical result of the parsed expression, applying left-to-right associativity for operators other than ** which will use right to left.
- **Result Display**: Output the computed result to the user in a clear, readable format.
- **Error Detection and Reporting**: Identify and report invalid expressions, including mismatched

parentheses, division by zero, invalid characters, missing operands, and incorrect operator sequences, with clear and informative error messages.

2.3 User characteristics

The primary users of the Arithmetic Expression Evaluator are team members and course instructors (TAs) evaluating the system. Users are expected to have a working understanding of basic arithmetic, operator precedence, and command-line interfaces. No expertise beyond standard mathematics and command-line interfacing is required to operate the program. Non-primary users are not expected to have knowledge of internal implementation. Because target audience is technically literate, concise, and informative error messages are preferred over lengthy explanatory output.

2.4 Constraints

The following constraints govern the design and implementation of the program:

- **Language Constraint**: The entire program must be implemented in C++ using object-oriented programming principles. No other programming languages are permitted.
- **Operator Scope**: The evaluator is restricted to the six specified operators (+, *, %, **) and unary + and -. Support for variables, functions, bitwise operators, or logical operators is explicitly out of scope.
- **Numeric Input**: The implementation must support integer constants.
- **Platform Constraint**: The program must compile and run correctly on personal computers and KU lab computers using standard C++ compilers.
- **Schedule Constraint**: The requirements phase must be completed by March 15, 2026, the design and architecture phase by April 5, 2026, and the final implementation, user manual, and test cases by May 7, 2026.

2.5 Assumptions and dependencies

The following assumptions have been made in defining the requirements for the Arithmetic Expression Evaluator:

- All team members possess sufficient working knowledge of C++ and object-oriented programming to contribute to the implementation.
- Input expressions will be provided as strings containing only the specified operators, numeric constants, parentheses, and whitespace. No other character types are assumed to be valid input.
- The host machine provides a functioning C++ compiler (g++) and a standard command-line environment. No additional software installation is required beyond the compiler toolchain.
- All team members will have continuous access to the shared GitHub repository & teamwork space on Microsoft Teams throughout the project life cycle.

The system has no external software dependencies beyond the C++ standard library. It does not depend on third-party parsing frameworks, mathematical libraries, or external APIs. The correctness of the C++ standard library on the target platform is assumed.

2.6 Requirements subsets

3. Specific Requirements

3.1 Functionality

3.1.1 <Functional Requirement One – Recognize Decimal Integer Literals>

Pattern: Optional sign (+ or -) followed by a non-empty sequence of decimal digits (0-9).

Valid Examples: 0, 123, +4096, -99

Invalid Examples: (empty), +, 12a3(letter breaks sequence, starts new token)

3.1.2 <Functional Requirement Two – Floating Point>

General Form: A fractional part OR an exponent part OR both. Must contain a decimal point, exponent, or both.

Rule Set:

1. Fractional Part: A sequence of digits, a decimal point (.), and a sequence of digits. At least one digit sequence must be present.

Valid: 1.0, .1, 1.

2. Exponent Part: e or E, optional sign (+/-), non-empty digit sequence.

Combined Valid Patterns:

[digits] . [digits] (ex. 3.14)

[digits] . [digits][exponent] (ex. 1.0e2, 1e-2)
[digits] [exponent] (ex. 1e5)
. [digits] [Exponent] (ex. 1E+2)
An optional leading sign (+/-) may precede the entire construct.
Valid Examples: 1., .10, -0.01, 1e2, 1.0E-2, +10.10E2
Invalid Examples: e2 (no digits before e), 1.2.3 (two points), 1e (no exponent digits)

3.1.3 <Functional Requirements – Token Output>

Upon recognition, the module shall return a TOKEN_NUMBER with two attributes:
1. Lexeme: The exact string found in the source code
2. Type: Sub-category (INT_DEC, FLOAT_DEC)

3.2 Use-Case Specifications

Use Case 1: Expression Evaluation: The primary actor is the user, who can enter an expression consisting of any positive/negative integer from 0-9 and any arithmetic operator (+, -, *, /, %, **). They will also be able to use any floating-point values (ex. 1.3 or 2E3). These values will be taken by the program, tokenized and parsed, and the result will be printed to the user. This should have error handling to ensure that any bad input won't crash the entire program. If any errors occur, a message should be printed to help the user resolve any issues that may be causing them.

3.3 Supplementary Requirements

Non-Functional Requirements

NFR1 (robustness): Shall never crash on malformed input; shall emit a TOKEN_ERROR and recover to a defined state

NFR2 (Performance): Shall process source code at a rate >= 100,000 lines/second on standard hardware.

NFR3 (integration): Shall provide a clean API:

Token getNextToken (input_stream)

get_next_token(stream: TextStream) -> Token

Assumptions

Assumption 1: The module receives characters sequentially

Assumption 2: Source character set is ASCII/UTF-8

4. Classification of Functional Requirements

Functionality	Type
Recognize Decimal Integer Literals	Essential
Recognize Decimal Floating-point Literals	Essential
Token Output	Essential
Robustness (No Crashing on bad input)	Essential
Performance	Essential
Integration (Clean API)	Essential

5. Appendices

This section may contain supporting information related to the Arithmetic Expression Evaluator, such as example inputs and outputs, test cases, or additional notes used during development. These materials are included only for reference and are not part of the main software requirements.